

PROYECTO ACADÉMICO · JAVA

TaskMaster

Proyecto de gestión de tareas desarrollado en Java para el entorno educativo de Desarrollo de Aplicaciones Multiplataforma.



¿Qué problema resuelve TaskMaster?

TaskMaster nace para dar respuesta a la necesidad de **organizar**, **priorizar** y **hacer seguimiento** de tareas de forma estructurada, eliminando la dispersión y mejorando la productividad personal y académica.

Organización

Estructura clara de tareas, categorías y estados.

Productividad

Flujo de trabajo optimizado desde la consola.

Seguimiento

Control del progreso y estado de cada tarea.



Tecnologías Utilizadas

El proyecto integra un stack tecnológico moderno y educativo, cubriendo desde el desarrollo backend hasta la documentación y el control de versiones.



Java

Lenguaje principal de desarrollo, orientado a objetos.



JUnit 5

Framework para pruebas unitarias y validación del código.



GitHub

Control de versiones y colaboración en equipo.



HTML & CSS

Maquetación y estilo de la interfaz web del equipo.



Javadoc

Documentación automática del código fuente.

```
public class Usuario { 3/ usages  👤 felipe

    public String getNombreUsuario() { no usages  👤 felipe
        return nombreUsuario;
    }

    public void setNombreUsuario(String nombreUsuario) { no usages  👤 felipe
        this.nombreUsuario = nombreUsuario;
    }

    public String getEmail() { 2 usages  👤 felipe
        return email;
    }

    public void setEmail(String email) { no usages  👤 felipe
        this.email = email;
    }

    public String getPassword() { no usages  👤 felipe
        return password;
    }

    public void setPassword(String password) { no usages  👤 felipe
        this.password = password;
    }
}
```

VISIÓN GENERAL

¿Qué hace TaskMaster?

TaskMaster es una aplicación de **consola interactiva** que permite gestionar todo el ciclo de vida de las tareas de forma intuitiva y estructurada.



Gestión de usuarios

Registro, autenticación y perfiles individuales.



Control de tareas

Creación, edición, eliminación y filtrado.

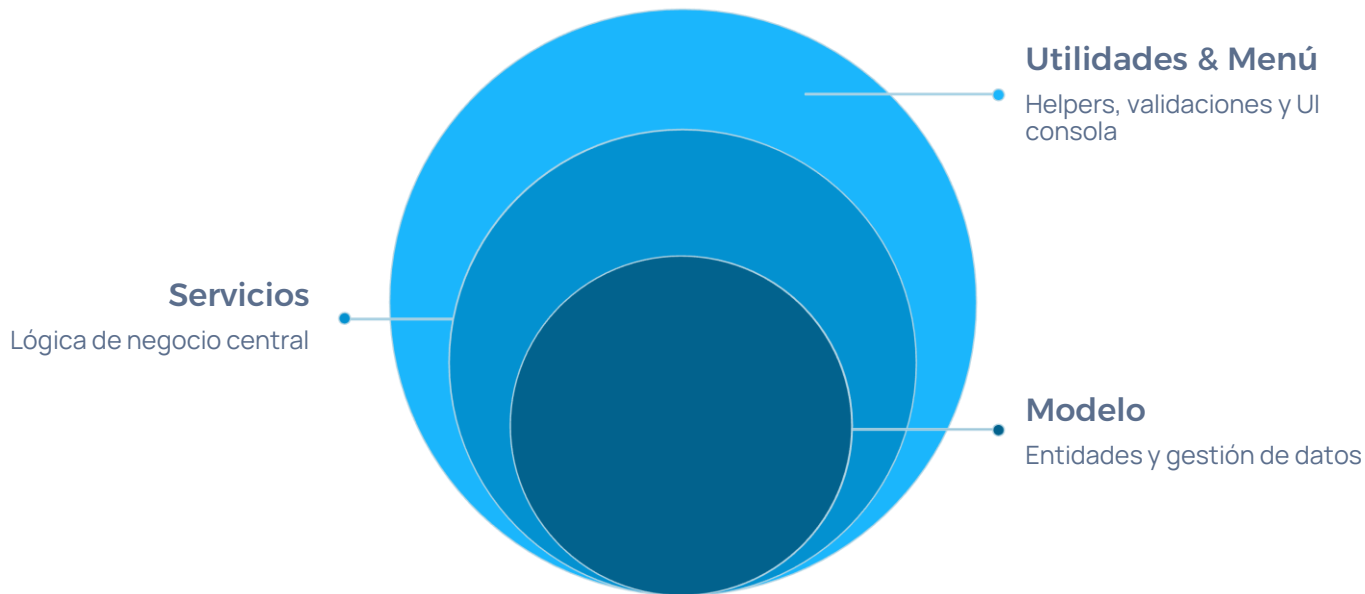


Categorías y estados

Organización por etiquetas y seguimiento del progreso.

Estructura del Sistema

El proyecto sigue una arquitectura en **capas bien diferenciadas**, promoviendo la separación de responsabilidades y facilitando el mantenimiento del código.



Cada capa tiene una responsabilidad única: el **Modelo** gestiona las entidades, los **Servicios** encapsulan la lógica, las **Utilidades** reutilizan funcionalidades comunes y el **Menú** interactúa directamente con el usuario.

Desarrollo de la Interfaz Web

Como complemento al proyecto, se diseñó una **página web del equipo** con dos vistas principales, aplicando técnicas modernas de diseño front-end.

Estructura visual

Cabecera y footer comunes, formulario de contacto funcional y diseño limpio con identidad propia.

Flexbox & Responsive

Maquetación flexible con Flexbox y adaptación a distintos dispositivos mediante diseño responsive.

Animaciones CSS

Transiciones y efectos visuales que mejoran la experiencia de usuario sin sacrificar rendimiento.

TaskMaster

[Inicio](#) [Contacto](#)

Panel de control de TaskMaster

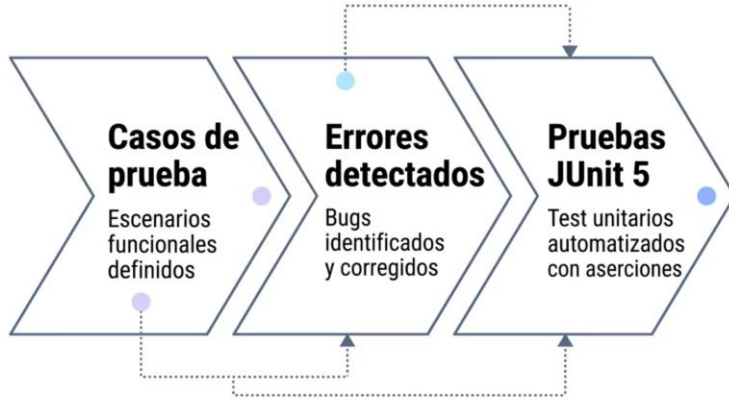
Gestiona tus tareas, usuarios, categorías y estados desde una interfaz clara y sencilla.

[Ver tareas](#)

[Crear tarea](#)

¡lo

Pruebas y Validación



Calidad garantizada

Se diseñaron **casos de prueba** para cubrir los flujos principales de la aplicación, detectando y corrigiendo errores antes de la entrega final. JUnit 5 permitió automatizar las pruebas unitarias con aserciones claras, validando el comportamiento esperado de cada método.



Resultado: Cobertura de los casos críticos del sistema con feedback inmediato durante el desarrollo.

Refactorización y Mejora del Código

A lo largo del proyecto se aplicaron técnicas de **refactorización continua** para elevar la calidad, legibilidad y mantenibilidad del código fuente.



Separación de responsabilidades

Cada clase asume un único rol dentro del sistema.



Clases auxiliares

Extracción de lógica repetitiva a utilidades reutilizables.



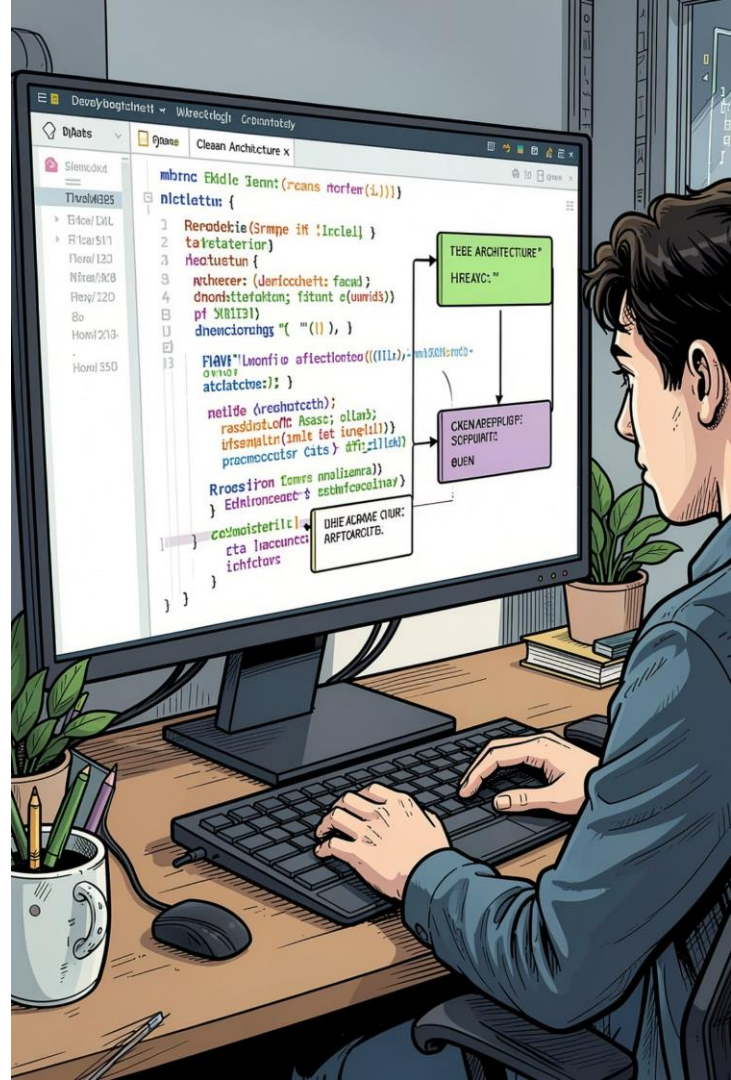
Javadoc completo

Documentación de clases, métodos y parámetros.



Calidad del código

Nombres significativos, estructura limpia y sin duplicidades.



Análisis de Riesgos

Durante el desarrollo se identificaron los principales **riesgos del proyecto** y se definieron medidas preventivas para minimizar su impacto.

1

Gestión del tiempo

Planificación con hitos semanales y revisión continua del avance.

2

Conflictos en Git

Comunicación previa a los merges y uso de ramas por funcionalidad.

3

Complejidad del código

Refactorización iterativa y revisión entre pares (code review).

4

Pruebas insuficientes

Integración temprana de JUnit y definición de casos de prueba desde el inicio.





VALORACIÓN FINAL

Conclusión Final

TaskMaster representa un **proyecto completo y bien estructurado** que integra conceptos clave del desarrollo de software: arquitectura en capas, pruebas, documentación y trabajo en equipo.

5

Tecnologías

Java, JUnit, GitHub, HTML, CSS

4

Capas del sistema

Modelo, Servicios, Utilidades, Menú

100%

Código documentado

Con Javadoc en todas las clases



Aprendizaje clave: El proyecto ha consolidado competencias técnicas y metodológicas esenciales para el desarrollo profesional de aplicaciones multiplataforma.